

3522 BULUT BİLİŞİM DERSİ
FINAL PROJESİ

Proje 7
IoT ve Akıllı Şehir Uygulaması

AWS IoT Core + DynamoDB ile Gerçek Zamanlı Sensör Verisi İşleme

Hazırlayan: Yusuf
Teslim Türü: Final Projesi Raporu

Proje deposu (tüm projeler): <https://github.com/ysftrv/bulut-bilisim-projeleri>
Bu projenin klasörü: <https://github.com/ysftrv/bulut-bilisim-projeleri/tree/main/proje-iot>
Video Linki = <https://youtu.be/NPjhMB4IPw4>

İçindekiler

İçindekiler.....	
1. Projenin Amacı ve Kapsamı	
2. Kullanılan Teknolojiler	
3. Sistem Mimarisi ve Çalışma Akışı	
4. Yapılan Adımlar	
4.1. AWS IoT Core'da Cihaz Tanımlama	
4.2. Güvenlik Politikası	
4.3. DynamoDB Tablosunun Oluşturulması	
5. Sensör Betiği (sensor.py)	
6. Verilerin Bulutta Toplanması ve Raporlanması	
7. Gerçek Zamanlı Görselleştirme (dashboard.py)	
8. Karşılanan Proje Çıktıları	
9. Güvenlik Önlemleri.....	
10. Karşılaşılan Zorluklar ve Öğrenilenler.....	
11. Sonuç.....	

1. Projenin Amacı ve Kapsamı

Bu proje, 3522 Bulut Bilişim dersi kapsamında verilen Proje 7 – IoT ve Akıllı Şehir Uygulaması başlığı altında gerçekleştirilmiştir. Projenin amacı, bir akıllı şehir sensöründen elde edilen verileri bulut ortamına aktarmak, bu verileri bulutta toplamak ve gerçek zamanlı olarak görselleştirmektir.

Fiziksel bir donanım kullanmak yerine, proje belgesinde de belirtildiği gibi, bir çevre sensörü Python ile simüle edilmiştir. Bu sensör; sıcaklık, nem ve hava kalitesi (AQI) verileri üretmektedir. Üretilen veriler MQTT protokolü ile Amazon Web Services (AWS) bulut platformundaki AWS IoT Core servisine gönderilmekte, Amazon DynamoDB veritabanında saklanmakta ve canlı bir grafikte görselleştirilmektedir.

Proje belgesinde Proje 7 için üç çıktı tanımlanmıştır: IoT cihazlarından veri toplama ve analiz, cihazların yönetimi ve raporlanması, ve gerçek zamanlı verilerin görselleştirilmesi. Bu rapor, bu üç çıktının nasıl karşılandığını adım adım açıklamaktadır.

2. Kullanılan Teknolojiler

Projede kullanılan başlıca teknolojiler aşağıdaki tabloda özetlenmiştir:

Bileşen	Kullanılan Teknoloji
Bulut Platformu	Amazon Web Services (AWS)
IoT Servisi	AWS IoT Core (cihaz yönetimi ve MQTT bağlantısı)
İletişim Protokolü	MQTT
Veritabanı	Amazon DynamoDB (veri toplama ve raporlama)
Programlama Dili	Python 3
AWS Kütüphaneleri	awsiodsdk (IoT bağlantısı), boto3 (DynamoDB)
Görselleştirme	Matplotlib (canlı grafik)
Sürüm Kontrolü	Git ve GitHub
Bölge (Region)	US East (N. Virginia) – us-east-1

AWS IoT Core, çok sayıda IoT cihazının bulut ile güvenli biçimde iletişim kurmasını sağlayan bir servistir. Cihazlar buluta MQTT protokolü üzerinden bağlanır. MQTT, az kaynak tüketen ve IoT uygulamalarında yaygın kullanılan bir mesajlaşma protokolüdür. Amazon DynamoDB ise hızlı ve ölçeklenebilir bir NoSQL veritabanıdır ve sensör verilerinin saklanması için kullanılmıştır.

3. Sistem Mimarisi ve Çalışma Akışı

Uygulamanın çalışma mantığı aşağıdaki adımlardan oluşmaktadır:

- Python ile yazılan sensör betiği, her iki saniyede bir sıcaklık, nem ve hava kalitesi verisi üretir.
- Üretilen veri, MQTT protokolü ile AWS IoT Core servisine güvenli (sertifika tabanlı) bir bağlantı üzerinden gönderilir.

3. Aynı veri, raporlama amacıyla Amazon DynamoDB veritabanına kaydedilir; böylece tüm ölçümler bulutta birikir.
4. Dashboard betiği, DynamoDB'deki verileri okuyarak canlı (gerçek zamanlı) bir grafik halinde görselleştirir.

Bu mimaride dikkat çeken nokta, verinin hem gerçek zamanlı olarak akması hem de kalıcı olarak saklanmasıdır. Tüm veri akışı bulut üzerinden gerçekleştiği için proje, bulut tabanlı bir IoT çözümü niteliği taşımaktadır.

4. Yapılan Adımlar

4.1. AWS IoT Core'da Cihaz Tanımlama

AWS IoT Core servisinde “akilli-sehir-sensor2” adında bir cihaz (Thing) oluşturulmuştur. Bu cihaz, simüle edilen sensörün buluttaki kimliğini temsil eder. Cihazın buluta güvenli bağlanabilmesi için AWS tarafından bir cihaz sertifikası, bir özel anahtar (private key) ve Amazon Root CA sertifikası üretilmiştir. Bu dosyalar sensör kodunda kullanılarak şifreli bir bağlantı sağlanmıştır.

4.2. Güvenlik Politikası

Cihaza, IoT işlemlerini gerçekleştirebilmesi için “akilli-sehir-policy” adlı bir politika tanımlanmıştır. Bu politika, cihazın bağlanma, yayın yapma ve abone olma izinlerini düzenler. Böylece cihaz yönetimi çıktısı karşılanmıştır.

4.3. DynamoDB Tablosunun Oluşturulması

Sensör verilerinin saklanması için Amazon DynamoDB'de “akilli-sehir-veriler” adlı bir tablo oluşturulmuştur. Tablonun bölüm anahtarı (partition key) olarak “cihaz_id” (String), sıralama anahtarı (sort key) olarak “zaman” (Number) tanımlanmıştır. Sıralama anahtarı sayesinde her ölçüm ayrı bir kayıt olarak saklanmaktadır.

5. Sensör Betiği (sensor.py)

Sensör betiği, hem AWS IoT Core'a MQTT ile veri yayınlar hem de aynı veriyi DynamoDB'ye kaydeder. Kodun temel bölümleri aşağıda gösterilmiştir:

```
# AWS IoT Core'a güvenli MQTT bağlantısı kurulur
mqtt_connection = mqtt_connection_builder.mtls_from_path(
    endpoint=ENDPOINT,          # cihaz veri adresi (IoT Core)
    cert_filepath=CERT,         # cihaz sertifikası
    pri_key_filepath=KEY,        # özel anahtar
    ca_filepath=CA,              # Amazon Root CA
    client_id="akilli-sehir-sensor2",
)
mqtt_connection.connect().result()

# DynamoDB tablosuna bağlantı
dynamodb = boto3.resource("dynamodb", region_name="us-east-1")
tablo = dynamodb.Table("akilli-sehir-veriler")

while True:
    veri = {
        "cihaz_id": "akilli-sehir-sensor2",
        "zaman": int(time.time()),
```

```

        "sicaklik": round(random.uniform(18, 35), 1),
        "nem": round(random.uniform(30, 80), 1),
        "hava_kalitesi": random.randint(20, 150),
    }
    # 1) MQTT ile IoT Core'a yayınla (gerçek zamanlı akis)
    mqtt_connection.publish(topic="akilli-sehir/sensor",
                           payload=json.dumps(veri),
                           qos=mqtt.QoS.AT_LEAST_ONCE)
    # 2) DynamoDB'ye kaydet (veri toplama / raporlama)
    tablo.put_item(Item=json.loads(json.dumps(veri, parse_float=Decimal))
    time.sleep(2)

```

Burada önemli bir teknik ayrıntı bulunmaktadır: DynamoDB, Python'un ondalık (float) sayı tipini doğrudan kabul etmez. Bu nedenle kayıt sırasında sayısal değerler Decimal tipine dönüştürülmüştür (parse_float=Decimal). Bu dönüşüm yapılmazsa “Float types are not supported” hatası alınır.

6. Verilerin Bulutta Toplanması ve Raporlanması

Sensör betiği çalıştırıldığında, üretilen her ölçüm hem AWS IoT Core'un MQTT test istemcisinde gerçek zamanlı olarak görüntülenebilmekte hem de DynamoDB tablosuna kaydedilmektedir. DynamoDB'nin “Explore items” ekranı incelendiğinde, her satırın bir sensör ölçümüne karşılık geldiği görülmektedir. Her kayıta cihaz kimliği, zaman damgası, sıcaklık, nem ve hava kalitesi değerleri yer almaktadır.

Böylece veriler yalnızca anlık olarak akmakla kalmamakta, aynı zamanda bulutta kalıcı olarak biriktirilip sonradan sorgulanabilir ve raporlanabilir hale gelmektedir. Bu durum, projenin veri toplama ve raporlama çıktısını karşılamaktadır.

7. Gerçek Zamanlı Görselleştirme (dashboard.py)

Dashboard betiği, DynamoDB'de biriken verileri okuyarak Matplotlib kütüphanesi ile canlı bir grafik oluşturur. Grafik, belirli aralıklarla (her iki saniyede bir) kendini yenileyerek yeni gelen verileri ekler. Kodun temel bölümü aşağıdadır:

```

dynamodb = boto3.resource("dynamodb", region_name="us-east-1")
tablo = dynamodb.Table("akilli-sehir-veriler")

def guncelle(frame):
    # DynamoDB'den son olcumlari oku
    items = tablo.scan().get("Items", [])
    items.sort(key=lambda x: int(x["zaman"]))
    items = items[-30:] # son 30 olcum

    zamanlar = [saat_bicimi(i["zaman"]) for i in items]
    sicaklik = [float(i["sicaklik"]) for i in items]
    hava = [float(i["hava_kalitesi"]) for i in items]

    ax.clear()
    ax.plot(zamanlar, sicaklik, marker="o", label="Sicaklik (C)")
    ax.plot(zamanlar, hava, marker="s", label="Hava Kalitesi (AQI)")
    ax.legend(); ax.grid(True)

# grafigi her 2 saniyede bir yenile
ani = animation.FuncAnimation(fig, guncelle, interval=2000)
plt.show()

```

Uygulama çalıştırılırken bir terminalde sensör betiği veri üretmeye devam ederken, ikinci bir terminalde dashboard betiği çalıştırılır. Açılan grafik penceresinde sıcaklık ve hava kalitesi değerlerinin zaman içindeki değişimi canlı olarak izlenebilir. Yeni veri geldikçe çizgiler ilerler. Bu, projenin gerçek zamanlı görselleştirme çıktısını karşılar.

8. Karşılanan Proje Çıktıları

Proje belgesinde Proje 7 için belirtilen üç çıktının bu projede nasıl karşılandığı aşağıda özetlenmiştir:

- IoT cihazlarından veri toplama ve analiz: Simüle edilen sensör, MQTT ile AWS IoT Core'a düzenli olarak veri göndermekte ve bu veriler DynamoDB'de toplanmaktadır.
- Cihazların yönetimi ve raporlanması: Sensör, IoT Core'da bir cihaz (Thing) olarak sertifika ve politika ile tanımlanmış; veriler DynamoDB'de saklanarak raporlanabilir hale getirilmiştir.
- Gerçek zamanlı verilerin görselleştirilmesi: Dashboard betiği, buluttaki verileri canlı güncellenen bir grafikte göstermektedir.

9. Güvenlik Önlemleri

Projede AWS IoT Core'a bağlanmak için kullanılan sertifika ve özel anahtar dosyaları, AWS hesabına erişim sağlayan hassas bilgilerdir. Bu dosyaların yanlışlıkla herkese açık GitHub deposuna yüklenmesini önlemek için .gitignore dosyasına ilgili uzantılar (*.pem, *.key, *.crt) eklenmiştir. Böylece gizli anahtarlar sürüm kontrolüne dahil edilmemiş ve güvenlik sağlanmıştır.

10. Karşılaşılan Zorluklar ve Öğrenilenler

Proje sürecinde öğrenilen başlıca konular şunlardır:

- AWS IoT Core'a bağlanmanın sertifika tabanlı (mTLS) bir güvenlik gerektirdiği ve bu sertifikaların kod tarafında doğru tanımlanması gerektiği öğrenilmiştir.
- DynamoDB'nin ondalık sayıları doğrudan kabul etmediği, bu nedenle Decimal dönüşümünün gerekli olduğu görülmüştür.
- Verinin buluta yazılması için farklı yöntemler bulunduğu; bu projede sade ve güvenilir olması açısından verinin doğrudan koddan DynamoDB'ye yazılmasının tercih edildiği öğrenilmiştir.
- Gizli anahtarların sürüm kontrolünden hariç tutulmasının güvenlik açısından önemli olduğu pekiştirilmiştir.

11. Sonuç

Bu projede bir akıllı şehir sensörü Python ile simüle edilmiş; ürettiği veriler MQTT protokolü ile AWS IoT Core'a gönderilmiş, Amazon DynamoDB'de toplanmış ve gerçek zamanlı bir grafikte görselleştirilmiştir. Böylece Proje 7 için tanımlanan üç çıktı da karşılanmıştır. Proje, IoT cihazlarının bulut servisleriyle nasıl entegre edildiğini ve verilerin uçtan uca (cihazdan görselleştirmeye) nasıl işlendiğini uygulamalı olarak göstermektedir.